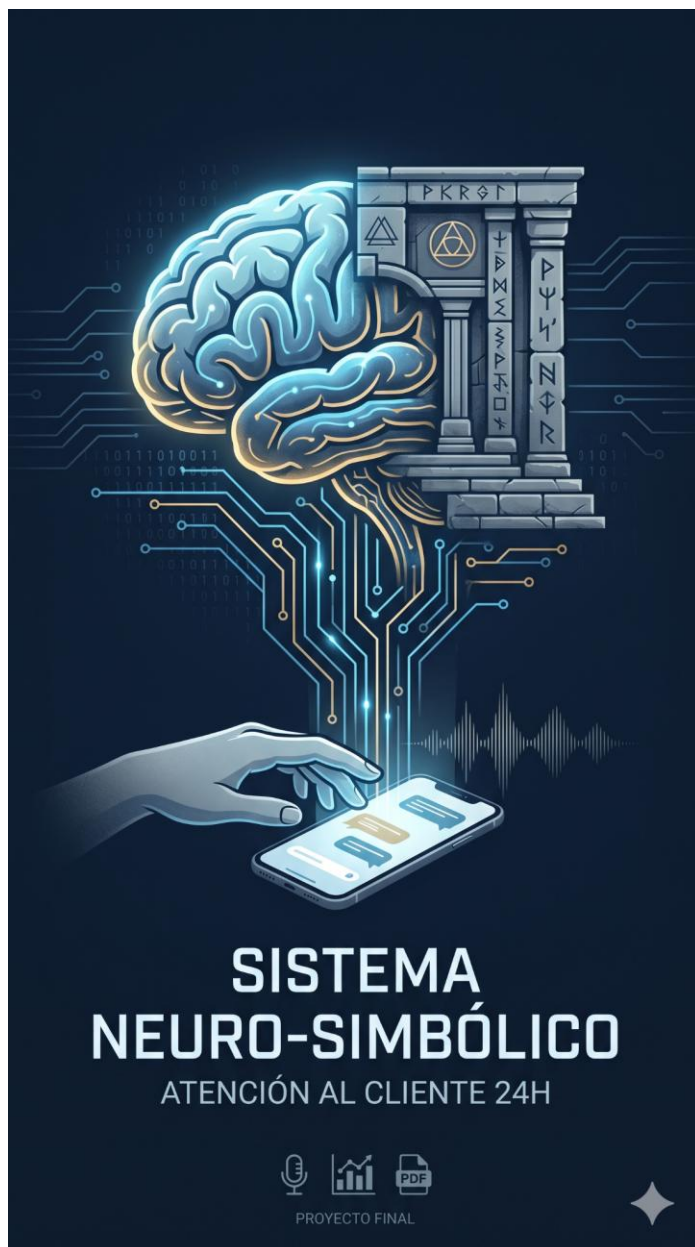


# Sistema Neuro-Simbólico de Atención al Cliente

## 1. Resumen Ejecutivo del Proyecto

El sistema desarrollado es una solución de IA Neuro-Simbólica que combina el aprendizaje profundo (Deep Learning) para la interpretación del lenguaje natural con la lógica simbólica (Sistemas Expertos) para la toma de decisiones empresariales. Esta arquitectura permite que el sistema no solo "entienda" lo que el usuario dice, sino que razone sobre ello basándose en reglas de negocio predefinidas y perfiles de cliente.



## 2. Arquitectura del Sistema y Fases

El proyecto lo he dividido en 5 fases críticas que fluyen de manera secuencial, coordinadas por el controlador central **main\_test.py**:

| Fase          | Nombre             | Función Principal                                                                                                               | Herramientas Clave              |
|---------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <b>Fase 1</b> | Captura y Limpieza | Obtención de audio (real o simulado) y normalización del texto (eliminación de tildes, caracteres especiales y limpieza regex). | SpeechRecognition, NLTK, Regex. |
| <b>Fase 2</b> | Análisis NLP       | Extracción de sentimiento y categorización de la incidencia mediante un modelo Transformer multilingüe.                         | Hugging Face (XLM-RoBERTa).     |
| <b>Fase 3</b> | Motor Experto      | Inferencia simbólica. Cruza el sentimiento y categoría con los datos del cliente (antigüedad) para decidir la respuesta.        | Experta (Basado en CLIPS).      |
| <b>Fase 4</b> | Salida de Voz      | Conversión de la resolución de texto a síntesis de voz (TTS) para cerrar el ciclo de interacción.                               | Pyttsx3.                        |
| <b>Fase 5</b> | Informe PDF        | Generación de un reporte técnico de la sesión con trazabilidad total de las decisiones tomadas.                                 | FPDF2.                          |

## 3. Análisis de Problemas: Jerarquía y Sentimientos

Uno de los retos técnicos más importantes durante el desarrollo fue el **conflicto de prioridades** en la categorización y cómo el sentimiento influye en el resultado final:

- **Problema de Duplicidad:** Inicialmente, las palabras clave de categorías como 'retraso\_envio' y 'producto\_roto' podían solaparse. Se resolvió implementando una **jerarquía de categorías** en la Fase 2, donde la información y el feedback positivo tienen prioridad para evitar falsos negativos en la atención al cliente.
- **Influencia del Sentimiento:** El modelo XLM-RoBERTa devuelve un valor numérico. El Sistema Experto utiliza este valor como un "Hecho" (Fact). Si el sentimiento es negativo ( $< 0.4$ ), el motor activa reglas de "Fidelización" o "Urgencia", cambiando la respuesta estándar por una de compensación, especialmente si el cliente es antiguo (VIP).
- **Robustez ante Datos Nulos:** Se implementó el manejo de clientes "Invitados" para que el sistema no colapse si el ID de usuario no existe, aplicando una política de cliente nuevo por defecto.

## 4. Modo de Ejecución

1. Abrir una ventana de PowerShell con el entorno activado.
2. Para instalar todo, solo ejecutar *pip install -r requirements.txt*
3. Ejecutar *python main\_test.py*.
4. Seleccionar el modo: **Manual** (Micrófono) o **Automático** (Simulación).
5. Identificar al cliente por ID.
6. En caso de seleccionar un cliente no existente se tomará como *Invitado*
7. En el modo manual se dictará el problema.
8. En el modo automático se tomarán frases ejemplo de las expuestas.
9. Observar el *Análisis Neuronal* (Fase 2).
10. El *Razonamiento Simbólico* (Fase 3) en consola.
11. La siguiente fase será la respuesta *Modo voz* de la resolución en la Fase 3
12. Finalizar con la opción 3 para exportar el informe PDF con el resumen de la sesión.

## 5. Funciones por archivo

### main\_test.py (Controlador Central)

Este archivo actúa como el **Orquestador del Sistema**, coordinando el flujo de datos entre todas las fases y gestionando el estado de la sesión.

| Función                    | Recibe                                                    | Realiza                                                                                                   | Se comunica con                                                         | Devuelve / Donde                                                          |
|----------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------|
| sonar_bips_llamada()       |                                                           | Activa una alerta sonora dual para indicar el inicio de una interacción.                                  | winsound (Beeps) y pyttsx3 (Voz).                                       |                                                                           |
| ejecutar_menu()            | Inputs del usuario por teclado (Opciones 1, 2 o 3 e IDs). | <b>Bucle Principal:</b> Gestiona la navegación, la selección de clientes y el historial de interacciones. | <b>Fases 1, 2, 3, 4 y 5:</b> Es el punto de unión de todos los módulos. | <b>A historial_sesion:</b> Almacena cada interacción en una lista en RAM. |
| ejecutar_menu() (Opción 3) | historial_sesion (Lista de dicts).                        | Finaliza el programa y solicita la exportación de datos acumulados.                                       | <b>Fase 5:</b> Llama a la generación del reporte.                       | <b>Archivo PDF:</b> Genera el documento físico en el directorio raíz.     |

- **Manejo de Errores (Robustez):** En la línea cliente = USUARIOS.get(id\_u, {"nombre": "Invitado", "meses": 0}) he implementado un **fallback**. Si el usuario introduce un ID erróneo, el sistema no se detiene; simplemente trata al cliente como un "Invitado" nuevo.
- **Arquitectura Desacoplada:** main\_test.py no sabe *cómo* se analiza el sentimiento o *cómo* se genera el PDF; solo sabe *cuándo* llamar a cada fase y qué datos pasarle. Esto facilita el mantenimiento del código.
- **Persistencia Temporal:** El historial solo vive en la memoria RAM durante la ejecución. Al seleccionar la opción 3, esos datos volátiles se convierten en un registro permanente (PDF).

### fase1\_captura.py (Captura y Preprocesamiento)

Este módulo es la puerta de entrada del sistema. Su responsabilidad es transformar la señal acústica (o simulada) en un formato de texto normalizado y libre de ruido para las fases posteriores.

| Función                   | Recibe                                                         | Realiza                                                                                                                                                          | Se comunica con                                                                            | Devuelve / Donde                                                              |
|---------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| limpieza_profesional()    | Texto crudo (String), booleanos para quitar números y acentos. | Convierte a minúsculas, elimina acentos (normalización NFD), suprime números y signos de puntuación mediante Regex, y normaliza espacios en blanco.              | Módulos internos re y unicodedata.                                                         | <b>A obtener_entrada_cliente:</b> String de texto limpio.                     |
| obtener_entrada_cliente() | Booleano modo_simulado (desde el menú principal).              | Gestiona la captura de datos. En modo simulado, elige una frase al azar de FRASES_PRUEBA. En modo real, gestiona la selección de hardware y la escucha de audio. | <b>Hardware:</b> Micrófono del sistema.<br><b>Módulo:</b> speech_recognition (Google API). | <b>A main_test.py:</b> El texto final normalizado tras pasar por la limpieza. |

- **Normalización de Datos:** Sin la limpieza (minúsculas y quitar acentos), el sistema de búsqueda por palabras clave de la Fase 2 fallaría ante variaciones ortográficas del usuario.

- **Gestión de Hardware:** La función de captura real incluye un paso de **calibración de ruido ambiental** (adjust\_for\_ambient\_noise) para mejorar la precisión del reconocimiento en entornos ruidosos.
- **Flexibilidad del Desarrollo:** El uso de una lista de **25 frases de prueba** permite realizar tests de estrés automáticos sin necesidad de hablar constantemente al micrófono, lo que aceleró la depuración de las reglas del Sistema Experto.
- **Eliminación de "Ruido":** Al aplicar expresiones regulares (re.sub), nos aseguramos de que caracteres especiales o números no interfieran con el análisis semántico del modelo Transformer en la siguiente fase.

## fase2\_analisis.py (Procesamiento de Lenguaje Natural)

En esta fase, el texto limpio se somete a un doble análisis: uno profundo mediante redes neuronales (Transformers) y otro estructurado mediante diccionarios jerárquicos.

| Función                        | Recibe                                         | Realiza                                                                                                                                                                                                             | Se comunica con                                                             | Devuelve / Donde                                                                  |
|--------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| pipeline("sentiment-analysis") | Parámetros del modelo (cardiffnlp).            | Carga en memoria el modelo <b>XLM-RoBERTa</b> , descargando los pesos y configurando el tokenizador multilingüe.                                                                                                    | <b>Hugging Face Hub:</b> Para la descarga y carga del modelo pre-entrenado. | <b>Variable Global:</b> Objeto analizador_sentimiento listo para procesar texto.  |
| analizar_incidencia_nlp()      | String de texto limpio (procedente de Fase 1). | 1. Calcula el sentimiento (Positivo/Negativo/Neutral) y su puntuación de confianza.<br><br>2. Clasifica el texto en una categoría específica comparando palabras clave contra 6 diccionarios temáticos priorizados. | <b>Modelo Transformer:</b> Para la inferencia de sentimiento.               | <b>A main_test.py:</b> Un diccionario con las claves "categoria" y "sentimiento". |

- **Modelo de Vanguardia (XLM-RoBERTa):** Se utiliza un modelo basado en la arquitectura Transformer, entrenado específicamente para detectar sentimientos en redes sociales, lo que lo hace muy robusto ante el lenguaje coloquial de los clientes.
- **Mapeo de Sentimiento:** Convierto las etiquetas del modelo (LABEL\_0, LABEL\_1, LABEL\_2) en un rango numérico continuo entre **-1 y 1**. Esto permite que el Sistema Experto en la Fase 3 pueda realizar cálculos matemáticos de prioridad basados en la intensidad del enfado o satisfacción.

- **Jerarquía de Categorización:** Este es un punto crítico el orden de los if/elif no es aleatorio:
  - Se prioriza **Información** y **Feedback Positivo** para evitar que palabras negativas genéricas clasifiquen erróneamente un agradecimiento como una queja.
  - La categoría **SAT** tiene prioridad sobre **Producto Roto** para capturar manuales y errores de software antes que daños físicos.
- **Estrategia de Seguridad:** Si ninguna palabra clave coincide, la función devuelve la categoría "desconocido". Esto garantiza que el flujo no se detenga y que el Sistema Experto pueda activar un protocolo de derivación manual.

### fase3\_motor.py (Motor de Inferencia Experta)

Este archivo utiliza el motor **Experta** (basado en el algoritmo Rete) para realizar una toma de decisiones lógica y explicable.

| Función / Clase                       | Recibe                                                           | Realiza                                                                                                                                           | Se comunica con                                                                                | Devuelve / Donde                                                                |
|---------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Clase MotorAtencion</b>            | Hechos iniciales (Incidencia, Cliente).                          | Ejecuta un motor de inferencia con reglas de <b>priorización, políticas VIP y seguridad.</b>                                                      | <b>Motor de Reglas:</b> Evalúa condiciones y dispara acciones basadas en salience (prioridad). | <b>Hecho Resolucion:</b> Modifica este hecho con la acción y el mensaje final.  |
| <b>procesar_con_sistema_experto()</b> | Diccionario datos_nlp (Fase 2), nombre y antigüedad del cliente. | 1. Instancia el motor y lo resetea.<br><br>2. Declara los hechos iniciales en la memoria de trabajo.<br><br>3. Ejecuta el razonamiento simbólico. | <b>Clase MotorAtencion:</b> Orquesta su ciclo de vida.                                         | <b>A main_test.py:</b> El String del mensaje_voz extraído del hecho Resolucion. |

- **IA Simbólica vs. Probabilística:** Mientras la Fase 2 es probabilística (predice sentimientos), esta fase es **determinista**. Si se cumplen las condiciones (Hechos), se dispara la regla sin ambigüedad.
- **Encadenamiento de Reglas:** El motor no toma una decisión de un solo paso:
  1. Primero deduce la **Prioridad** (Bloque A) basándose en la categoría y la intensidad del sentimiento.

2. Después, esa prioridad se combina con el perfil del **Ciente** para aplicar la **Política** adecuada (Bloque B).
- **Gestión VIP y Fidelización:** La regla politica\_vip demuestra inteligencia de negocio al aplicar el hecho Resolución con una acción de "BONO\_FIDELIDAD" solo si la antigüedad supera los 24 meses, independientemente de la queja.
  - **Uso de Saliency:** Uso la propiedad saliency para controlar el orden de ejecución. Las reglas de prioridad tienen saliency alto (100) para ejecutarse antes que las de resolución.
  - **Capas de Seguridad:** El sistema incluye un "paracaídas" (seguridad\_final) con saliency negativo (-100) que garantiza que el usuario siempre reciba una respuesta, incluso si ninguna otra regla coincide.
  - **Parche de Compatibilidad:** Corrección para collections.Mapping, asegurando que la librería Experta funcione correctamente en versiones modernas de Python (3.10+).

#### fase4\_voz.py (Salida de Voz / TTS)

Este archivo utiliza la librería pyttsx3, la cual destaca por ser un motor de síntesis de voz que funciona de manera local (offline), garantizando la privacidad y rapidez del sistema.

| Función             | Recibe                                                            | Realiza                                                                                                                                                                                                                       | Se comunica con                                                                                | Devuelve / Donde                                         |
|---------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| hablar_resolucion() | String con el mensaje de resolución final (procedente de Fase 3). | 1. Inicializa el motor de síntesis de voz.<br><br>2. Configura la velocidad del habla (rate).<br><br>3. Filtra y selecciona automáticamente una voz en español del sistema operativo.<br><br>4. Procesa y reproduce el audio. | <b>Drivers de Audio del SO:</b><br>Interactúa con las voces instaladas en Windows/Linux/macOS. | <b>Altavoces del sistema:</b><br>Salida física de audio. |

- **Sintetizador Offline:** pyttsx3 no requiere conexión a internet (a diferencia de otras APIs), lo que asegura que el asistente virtual siempre pueda responder de forma instantánea.
- **Adaptabilidad Multilingüe:** El código incluye un bucle que recorre las voces disponibles en el ordenador del usuario para seleccionar automáticamente una con etiqueta "spanish" o "ES", evitando que el sistema hable español con acento extranjero por defecto.

- **Control del Flujo (runAndWait):** Bloquea la ejecución del programa hasta que el asistente termina de hablar, evitando que el menú principal de `main_test.py` aparezca antes de que el cliente haya escuchado la solución.
- **Naturalidad:** Se ha configurado el parámetro `rate` a 150. Esto se hizo tras varias pruebas para asegurar que la velocidad de locución sea clara y profesional, no demasiado lenta ni robóticamente rápida.

### fase5\_informe.py (Generación de Reportes PDF)

Este archivo utiliza la librería `fpdf2` para construir un documento técnico que ofrece trazabilidad total sobre el comportamiento del sistema experto.

| Función / Clase              | Recibe                                                                          | Realiza                                                                                                                                                                                                             | Se comunica con                                                                      | Devuelve / Donde                                                                      |
|------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Clase PDF_Reporte</b>     | Métodos internos de <code>FPDF</code> .                                         | Define la identidad visual del documento, estableciendo el <b>encabezado corporativo</b> (azul con títulos blancos) y el <b>pie de página</b> con numeración automática.                                            | <b>Librería FPDF:</b> Sobrescribe métodos estándar para personalizar el diseño.      | <b>Plantilla de página:</b> Estructura visual de cada folio del PDF.                  |
| <b>generar_pdf_resumen()</b> | Lista <code>historial_sesion</code> (procedente de <code>main_test.py</code> ). | 1. Configura el documento en formato <b>A4 apaisado</b> (Landscape).<br><br>2. Calcula dinámicamente la altura de las filas para evitar solapamientos de texto.<br><br>3. Genera una tabla con 5 columnas técnicas. | <b>Módulo datetime:</b> Para estampar el nombre del archivo con fecha y hora exacta. | <b>Archivo físico (.pdf):</b> Genera el documento en el directorio raíz del proyecto. |

- **Trazabilidad y Explicabilidad (XAI):** El sistema no es una "caja negra". Permite a un auditor revisar qué dijo el cliente, qué categoría detectó el modelo neuronal y qué solución exacta aplicó el motor de reglas.
- **Diseño Responsivo de Tabla:** El código utiliza `get_string_width` para medir el texto antes de escribirlo, calculando un `alto_fila` proporcional. Esto garantiza que las celdas se ajusten perfectamente sin importar la longitud de la respuesta.

- **Control de Coordenadas:** Debido a que `multi_cell` desplaza el cursor verticalmente, el sistema debe reposicionar manualmente el cursor para completar el dibujo de las columnas de la derecha en la misma fila.
- **Gestión de Saltos de Página:** Si detecta que la siguiente fila superará los 185 mm de altura, añade automáticamente una página nueva para evitar que la tabla se corte.
- **Nomenclatura Única:** El archivo se genera con el formato `AñoMesDía_HoraMinutoSegundo`. Esto evita la pérdida de datos por sobreescritura y permite un archivo cronológico de todas las interacciones del sistema experto.

## 6. Desglose de librerías instaladas

- **experta:** El motor de lógica simbólica (Sistema Experto).
- **transformers & torch:** Para cargar y ejecutar el modelo de IA **XLNet** de Hugging Face.
- **SpeechRecognition & PyAudio:** Necesarios para capturar y reconocer la voz a través del micrófono.
- **pyttsx3:** El motor de síntesis de voz (Text-to-Speech) que funciona offline.
- **nltk:** Herramientas de procesamiento de texto para la limpieza y normalización.
- **fpdf2:** La librería para generar el informe técnico en PDF.
- **protobuf, sentencepiece, tiktoken:** Librerías de soporte necesarias para que el modelo Transformer funcione correctamente en versiones modernas de Python.